

MATA54 - Estruturas de Dados e Algoritmos

II

Departamento de Ciência da Computação
Instituto de Matemática e Estatística
UFBA

Prof. George Lima, Ph.D.

Filtros e assinaturas

Motivação: filtragem eficiente de informação

Aplicações: busca aproximada

Pode-se codificar valores de campos, agrupando-os, de forma a melhorar a eficiência para busca.

- ▶ Assinaturas são códigos
- ▶ Diferentes valores podem possuir o mesmo código
- ▶ Mesmos valores possuem o mesmo código

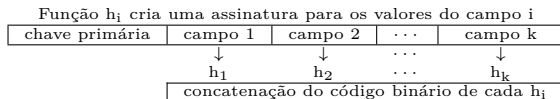
Estrutura de dados a serem estudadas

- ▶ Árvores de assinaturas (chaves secundárias)
- ▶ Páginas de assinaturas (chaves secundárias)
- ▶ Filtros de Bloom (chave primária)

Assinatura de registros

Codificação para valores

Uso de funções hashing

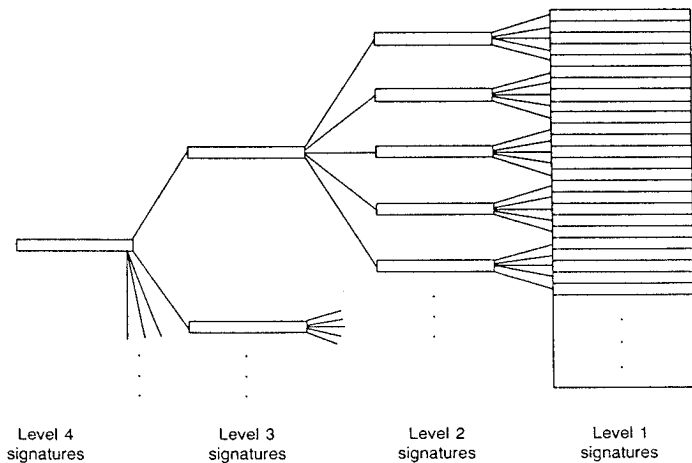


Exemplo com dois campos, x_1 e x_2

$$h_1(x_1) = 1 + x_1 \bmod 7 \quad h_2(x_2) = \begin{cases} 1 & \text{se } x_2 \leq 1000 \\ 2 & \text{se } 1000 < x_2 \leq 5000 \\ 3 & \text{se } x_2 > 5000 \end{cases}$$

Árvores de assinaturas

Superposição de assinaturas: “OU” bit-a-bit



Retirado de [Tharp]

Ilustração - Adaptada de [Tharp]

Cadastro de funcionários

- ▶ Ligar bit i ($1 \leq i \leq 8$): $h_i = 1 + \text{cargo} \bmod 8$;
- ▶ Ligar bit i ($9 \leq i \leq 13$):

$$h_i = \begin{cases} 9 & \text{se dept inicia com 'A-E'} \\ 10 & \text{se dept inicia com 'F-J'} \\ 11 & \text{se dept inicia com 'K-M'} \\ 12 & \text{se dept inicia com 'N-R'} \\ 13 & \text{se dept inicia com 'S-Z'}. \end{cases}$$

- ▶ Ligar bit i ($14 \leq i \leq 17$): $h_i = 14 + (\text{gerencia}) \bmod 4$;
- ▶ Ligar bit i ($18 \leq i \leq 23$):

$$h_i = \begin{cases} 18 & \text{se salario} \leq 2.000 \\ 19 & 2.000 < \text{salario} \leq 5.000 \\ 20 & 5.000 < \text{salario} \leq 8.000 \\ 21 & 8.000 < \text{salario} \leq 12.000 \\ 22 & 12.000 < \text{salario} \leq 15.000 \\ 23 & \text{salario} > 15.000. \end{cases}$$

Exemplo de árvores de assinaturas

								Address				
								s_1	s_2	s_3	s_4	
								00010000	01000	1000	010000	0
								00000010	00001	0010	010000	1
10010010	01011	1010	010011					00000000	00000	0000	000000	2
								10000000	00010	0010	000010	3
								00010000	00010	1000	000001	4
								00010000	00001	0010	000010	5
10110010	01011	1011	111111	10010010	01011	1010	000111	10000000	00010	0010	000001	6
								00010000	00010	0010	000010	7
								00000010	01000	1000	000100	8
								00010000	01000	1000	000001	9
								00000010	00010	0010	000001	10
								10000000	00010	0010	100000	11
10110010	01010	1011	101001					00010000	00010	0010	000001	12
								00100000	01000	0001	000001	13
								00010000	01000	1000	001000	14
								00000010	10000	0010	010000	15
								00000100	00010	0010	001000	16
00010110	10010	1010	011001					00010000	10000	0010	010000	17
								00000000	00000	0000	000000	18
00110111	11011	1110	011101					00010000	10000	1000	000001	19
								00000010	10000	0010	000100	20
								00000001	00010	0010	001000	21
00110011	11011	1110	011101					00010000	00010	0010	000001	22
								00100000	01000	0100	010000	23
								00000010	00001	1000	010000	24
Level 3		Level 2				Level 1						

Level 3

Level 2

Level 1

Retirado de [Thap]

Busca de informação

Pesquisas conjuntivas

Campo i satisfaz critério A “e” Campo j satisfaz critério B

Funções h_i e h_j consideradas conjuntamente						
chave primária	...	campo i	...	campo i	...	campo k
		↓	...	↓		
		h_i	...	h_j		
0...	...	010...	...	...	010...	...0

Pesquisas disjuntivas

Campo i satisfaz critério A “ou” Campo j satisfaz critério B

Funções h_i e h_j consideradas separadamente						
chave primária	...	campo i	...	campo i	...	campo k
		↓	...	↓		
		h_i	...	h_j		
0...	...	010...	...			...0
0...				...	010...	...0

Exemplos de buscas

- ▶ Quais funcionários trabalham no departamento de Dados?
 - ▶ Bit 9 ligado e os demais desligados.
- ▶ Quais funcionários que trabalham no departamento de Dados e ganham mais que 15.000,00?
 - ▶ Bits 9 e 23 ligados e os demais desligados.
- ▶ Quais funcionários que trabalham no departamento de Dados e ganham mais que 9.000,00?
 - ▶ Bits 9 e 21 ligados e os demais desligados; Bits 9 e 22 ligados e os demais desligados; Bits 9 e 23 ligados e os demais desligados;

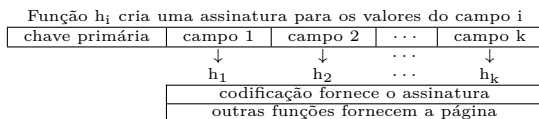
Observações sobre Árvore de assinaturas

- ▶ Quanto maior o número de campos em buscas conjuntivas, mais eficiente tende ser a busca.
 - ▶ O oposto do que ocorre com arquivos invertidos, por exemplo.
- ▶ O número de bits associado ao campo fornece seu “peso” como descritor. Maior número de bits, mais discriminado será o campo.
 - ▶ Tal característica não está presente em outros métodos, a exemplo de arquivos invertidos.
- ▶ Quanto o valor do campo do registro muda, a árvore deve ser atualizada. Esta atualização pode ser custosa. Por que?
- ▶ Aplicações práticas tipicamente usam assinaturas variando de 100 a 200 bits e agrupamentos de assinaturas consideram não menos que 100 por grupo [Tharp].

Página de assinaturas

Caracterização

Assinaturas de registros formam a assinatura da página onde eles residem. Registros são mapeados em páginas de acordo com seus conteúdos, através de funções hash.



Exemplo com dois campos, x_1 e x_2

Mapeamento usando 32 bits

$$h_1(x_1) = x_1 \bmod 7 \quad h_2(x_2) = \begin{cases} 0 & \text{se } x_2 \leq 1000 \\ 1 & \text{se } 1000 < x_2 \leq 5000 \\ 2 & \text{se } 5000 < x_2 \leq 6000 \\ 3 & \text{se } x_2 > 6000 \end{cases}$$

Ilustração - Adaptada de [Tharp]

Cadastro de funcionários

Número da página (16 páginas):

► Bit 1: $h_1 = \text{cargo} \bmod 2$;

► Bit 2:

$$h_2 = \begin{cases} 0 & \text{se dept inicia com 'A-M'} \\ 1 & \text{se dept inicia com 'N-Z'} \end{cases}$$

► Bit 3: $h_3 = (\text{gerencia}) \bmod 2$;

► Bit 4 ($18 \leq i \leq 23$):

$$h_i = \begin{cases} 0 & \text{se salario} \leq 8.000 \\ 1 & \text{salario} > 8.000. \end{cases}$$

Exemplo de página de assinaturas

Page signatures				Record signatures				Page	number
00010000	01000	1000	010000	00010000	01000	1000	010000	0000	0
00110000	01000	1001	000001	00010000 00100000	01000 01000	1000 0001	000001 000001	0001	1
00110010	11000	1110	011000	00010000 00100000 00010000 00000010	01000 01000 10000 10000	1000 0100 0010 0010	001000 010000 010000 010000	0010	2
00010000	10000	1000	000001	00010000	10000	1000	000001	0011	3
00000000	00000	0000	000000						4
00010000	00011	0010	000011	00010000 00010000	00001 00010	0010 0010	000010 000001	0101	5
00000010	00001	0010	010000	00000010	00001	0010	010000	0110	6
10000010	00010	0010	000001	10000000 00000010	00010 00010	0010 0010	000001 000001	0111	7
00000000	00000	0000	000000						8
00000010	10000	0010	000100	00000010	10000	0010	000100	1001	9
00000000	00000	0000	000000						10
00000010	01000	1000	000100	00000010	01000	1000	000100	1011	11
10000011	00011	1010	111000	10000000 00000010 00000001	00010 00001 00010	0010 1000 0010	100000 010000 001000	1100	12
10001000	00010	0010	000010	10000000 00001000	00010 00010	0010 0010	000010 000010	1101	13
00000100	00010	0010	001000	00000100	00010	0010	001000	1110	14
00010000	00010	1010	000001	00010000 00010000	00010 00010	0010 1000	000001 000001	1111	15

Exemplos de buscas

- ▶ Quais funcionários trabalham no departamento de Dados?
 - ▶ Oito páginas são relevantes: ‘?0??’;
 - ▶ A assinatura da página deve conter o bit 9 ligado.
- ▶ Quais funcionários que trabalham no departamento de Dados e ganham mais que 15.000,00?
 - ▶ Quatro páginas são relevantes: ‘?0?1’;
 - ▶ A assinatura da página deve conter os bits 9 e 23 ligados.

Filtro de Bloom

Filtro de Bloom

Caracterização e aplicações

- ▶ Quando se quer evitar o esforço de busca se a probabilidade do item a ser pesquisado satisfazer ao critério de busca é alta
 - ▶ Cliente x está negativado?
 - ▶ Site com IP y é suspeito?
- ▶ Uso de funções hashing para construir vetor binário com indicadores
 - ▶ Se o filtro indicar positivo, deve-se verificar (busca do item é necessária)
 - ▶ Se o filtro indicar negativo, evita-se a busca do item

Filtro de Bloom

- ▶ n funções hashing aplicadas às chaves primárias
- ▶ cada função hashing h_i indica se o bit $B[i]$ deve ser ligado
- ▶ o filtro possui tamanho m
 - ▶ se $h_i(x) = j$, $B[j] = 1$ indica pertinência de x com relação à h_i

Exemplo para $n = 2$:

$$h_1(x) = x \bmod 16 \quad h_2(x) = 2 + x \bmod 14$$

Tal que $x \in \{99, 43, 73, 35\}$ está com crédito negativado.
Deseja-se saber se existe elemento em
 $\{63, 17, 45, 23, 79, 51, 33, 59, 67, 95\}$ que está com crédito negativado.

B:	0	0	0	1	0	1	0	0	0	1	0	1	0	0	0	0
h:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Projeto do filtro de Bloom

Parametros

- ▶ m : número de bits do filtro
- ▶ n : número de funções hashing
- ▶ k : número de registros representados pelo filtro
- ▶ α : % esperado do universo de registros que se espera satisfazer os critérios do filtro

Deseja-se saber qual a probabilidade de um **falso positivo** assumindo que as funções hashing distribuem uniformemente as chaves no intervalo $[0, m - 1]$

- ▶ Probabilidade de um bit do filtro ser ligado (p_1) ou permanecer desligado (p_0) por uma função aplicada a um registro

$$p_1 = \frac{1}{m} \quad p_0 = 1 - \frac{1}{m}$$

Projeto do filtro de Bloom

- Probabilidade de um bit estar desligado depois de se aplicar as n funções hashing a um registro (**assumindo independência**)

$$p_0^n = \left(1 - \frac{1}{m}\right)^n$$

- Probabilidade de um bit estar desligado depois de se aplicar as n funções hashing aos k registros

$$p_0^{nk} = \left(1 - \frac{1}{m}\right)^{nk}$$

- Probabilidade de um bit estar ligado depois de se aplicar as n funções hashing aos k registros

$$1 - p_0^{nk} = 1 - \left(1 - \frac{1}{m}\right)^{nk}$$

Projeto do filtro de Bloom

Ao verificar se um registro está representado no filtro, qual a probabilidade de falso positivo?

- Probabilidade de n bits estarem ligados depois de se aplicar as n funções hashing a um registro a ser verificado

$$P = (1 - p_0^{nk})^n = \left(1 - \left(1 - \frac{1}{m}\right)^{nk}\right)^n$$

- Probabilidade de falso positivo para algum registro considerando que α é a porção de registros que se esperada estar representada

$$F = (1 - \alpha)P$$

- Os valores de n , m devem ser escolhidos para valores de k e α considerados representativos para o cenário da aplicação

Resumo

- ▶ Processamento de informação usando chave secundária requer apoio a recuperação de sub-conjuntos de registros
 - ▶ Assinatura digital cria descritores que podem ser usados para construção de filtros eficientes
 - ▶ Hashing tem papel importante para criação dos filtros
 - ▶ Árvore de assinaturas oferece estrutura hierárquica, mas independente da localização dos registros
 - ▶ Página de assinaturas requer autonomia para definir localização dos registros ou uso de uma camada extra de indireção
- ▶ Filtros de Bloom, quando maior probabilidade está associada a não encontrar o que se procura